MATPMD4: Optimisation Assignment

Student ID: 3539054

April 8, 2026

1 Maximising a Function

1.1 Problem Definition

The primary objective is to find the global maximum of the continuous function $f(x, y, z)$, subject to strict boundary constraints.

Objective Function:

Maximise $f(x, y, z)$ where:

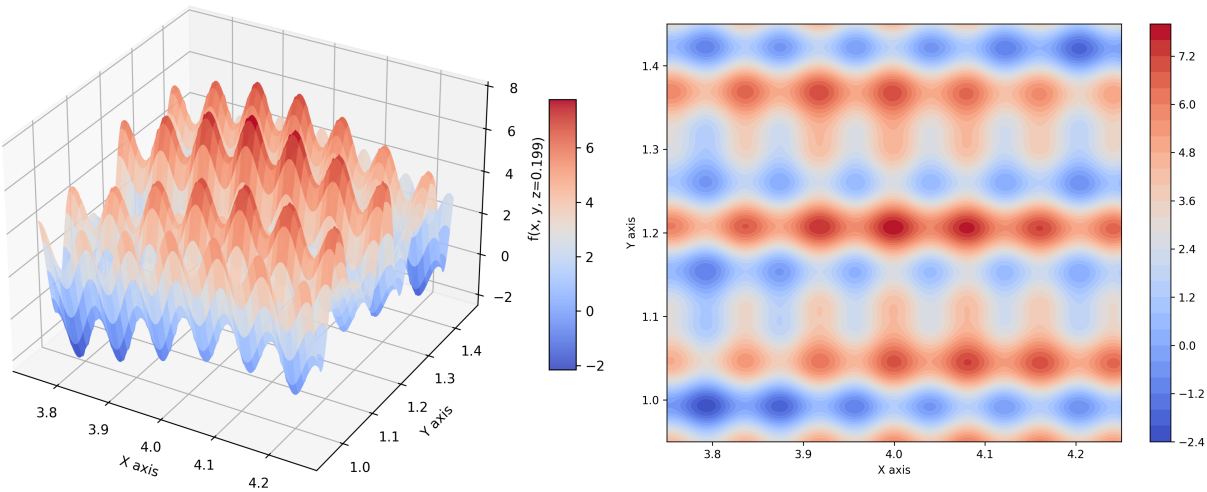
$$\begin{aligned} f(x,y,z) = &-e^{-(x-0.55z)^2}\cos(81(x+z+0.12y)) + \sin(44(y+0.03(x-z))) \\ &- \cos(69\sin(xz+0.07y)) - \sin(1.32(x-z)) - 0.32(z+0.22xz)^2 e^{\sin(63(z-x))} \\ &+ \cos(77x) + \sin(71z) - e^{-y^2}\cos(75y) + \sin(43y) + \cos(73y) + 0.05(x^2+y^2+z^2) \end{aligned}$$

Constraints:

$$\begin{aligned} 0 \le x \le 5 \\ 0 \le y \le 5 \\ 0 \le z \le 5 \end{aligned}$$

Landscape Visualisation:

Since the objective function relies on three independent variables (x, y, z), the complete search space is 4-dimensional and impossible to plot. To visualise the terrain, take a *slice* of the space by holding one variable constant. Figure 1 fixes $z = 0.199$ and zooms into a narrow, 0.5×0.5 window for x and y. Looking closely at this small area shows its real complexity. The surface is full of sharp peaks and steep valleys caused by overlapping trigonometric terms. This multimodal landscape explains why simple gradient-based methods would easily get stuck. It also shows why we need a strong random algorithm to explore this space effectively.



(a) 3D surface plot

(b) 2D contour map

Figure 1: Visualisations of the objective function's fitness landscape at the fixed slice $z = 0.199$. The view is zoomed in at $3.75 \leq x \leq 4.25$ and $0.95 \leq y \leq 1.45$

1.2 Algorithm Selection and Justification

Knowing that the objective function is complicated, a robust search strategy is needed. Methods like Hill Climbing or basic Simulated Annealing only keep one solution and try to make it better step by step. In a topography with many false peaks, these single-state methods often get stuck at a local maximum and cannot escape [1]. To overcome this, a **population-based method** is better. Population-based algorithms keep a variety of possible solutions. This allows exploration of different areas at the same time. By evaluating many options together, the risk of settling on a local maximum is reduced.

To solve this specific problem, a $(\mu + \lambda)$ **Evolution Strategy** (*ES*) was selected. *ES* is a population-based metaheuristic explicitly designed for continuous, real-valued $(\mathbb{R})$ optimisation. It was preferred because:

- **Real-Valued $(\mathbb{R})$ Population:** A parent population of size μ is maintained. Since the algorithm's parameters are real numbers, it can work with the coordinates (x, y, z).
- **Gaussian Mutation:** Parents produce λ offspring by adding random noise to their coordinates. This noise follows a normal distribution, which helps balance two approaches: making small, frequent adjustments to climb local peaks (*exploitation*) and taking occasional large jumps to escape traps (*exploration*).
- **Selection (Elitism):** The μ parents and λ offspring are combined, and the fittest μ individuals survive to the next generation. This $(\mu + \lambda)$ selection makes sure that the best solutions are preserved.

1.3 Algorithm Implementation and Results

Constraint Handling:

To enforce the strict boundary conditions $(x, y, z \in [0, 5])$, a clamping function was implemented. During reproduction, if a mutation moved a child's coordinate outside the allowed range, it was adjusted back to the closest valid boundary.

```
def clamp(value, min_bound=0.0, max_bound=5.0):
    return max(min_bound, min(value, max_bound))

# ... [Code skipped: inside the run_evolution_strategy loop] ...

# Gaussian mutation
child = {
    'x': clamp(parent['x'] + random.gauss(0, mutation_step_size)),
    'y': clamp(parent['y'] + random.gauss(0, mutation_step_size)),
    'z': clamp(parent['z'] + random.gauss(0, mutation_step_size))
}

```

Listing 1: Constraint Handling and Mutation Logic

Hyperparameter Optimisation:

Stochastic algorithms use randomness, so running the program only once doesn't ensure the best result. To find the most reliable settings, different combinations of parent sizes (μ) and offspring sizes (λ) were tested. The tests showed that using a small group of 10 parents to create a large pool of 300 offspring ($\mu = 10, \lambda = 300$) gave the best results. This approach allowed the algorithm to quickly get rid of poor options while still exploring nearby peaks effectively.

```
mu_options = [10, 30, 50]
lambda_options = [50, 150, 300]
trials_per_combo = 3

for mu, lambda_ in itertools.product(mu_options, lambda_options):
    for trial in range(trials_per_combo):
        result = run_evolution_strategy(mu, lambda_)
        if result['fitness'] > best_overall['fitness']:
            best_overall = result
            best_params = {'mu': mu, 'lambda': lambda_}

```

Listing 2: Automated grid search implementation for hyperparameter optimisation across varying parent (μ) and offspring (λ) population sizes.

Final Results:

Using the optimised parameters, the algorithm discovered the following global maximum:

- **Optimal Fitness:** 7.982176
- **Coordinates:** $x = 3.998687, y = 1.206588, z = 0.200044$

2 Distribution Network

2.1 Problem Definition

The scenario involves a Multi-Depot Heterogeneous Fleet Vehicle Routing Problem (*MD-HFVRP*) on a 2D plane. To speed up mathematical calculations during the algorithm's process, the physical coordinates (x_i, y_i) of the two warehouses $(W1, W2)$ and the 23 stores were re-organised into a `numpy` array.

First, a 25×25 **Euclidean distance matrix** was generated, where the distance between any two nodes is $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. By calculating these distances in advance, the routing algorithm can quickly look them up in constant time, rather than having to recalculate the distances repeatedly during the simulation.

Secondly, the coordinates were transformed to find the **polar angle** (θ) of each store relative to both $W1$ and $W2$ (Figure 2). By doing this before starting the algorithm, we set the stage for using an *angular sweep* heuristic. This heuristic avoids extreme angle differences, helping the vehicles to create more efficient routes instead of zig-zagging.

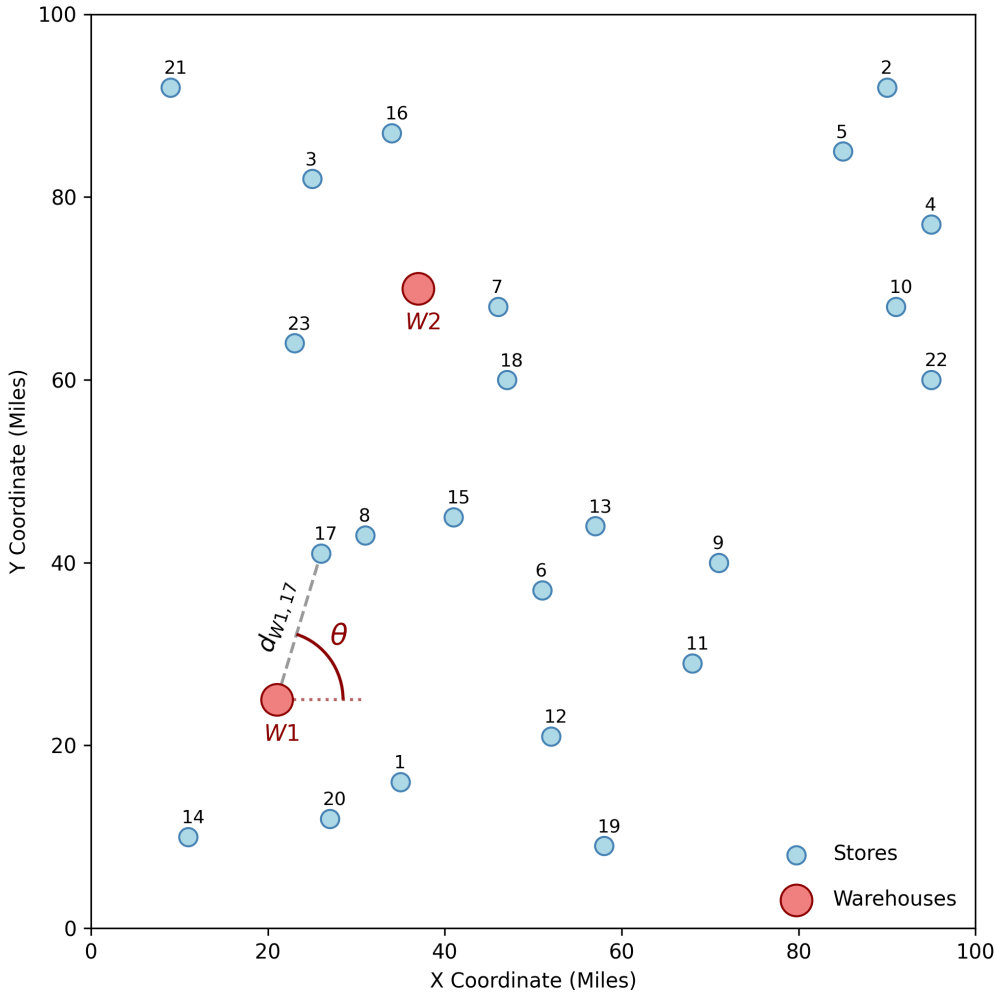


Figure 2: The supermarket distribution network featuring 23 stores and 2 warehouses $(W1, W2)$. It illustrates the extraction of Euclidean distance $(d_{W1,17})$ and polar angle (θ).

2.2 Algorithm Selection and Justification

To solve this routing problem, both **Genetic Algorithms** (*GA*) and **Ant Colony Optimisation** (*ACO*) were considered. *GAs* can find good solutions, but they struggle with routing problems that have strict rules. Research shows that using standard genetic crossover on a routing sequence often creates invalid routes, such as visiting one store multiple times while missing others [1]. Although special techniques like Partially Mapped Crossover (PMX) can help avoid duplication, mixing routes makes it hard to maintain strict limits on vehicle capacities (5 stores for Vans, 18 for Lorries). As a result, a *GA* would need costly repair processes in every generation to fix overloaded vehicles.

***ACO* was chosen** as the best architecture for this task. It was originally designed for routing networks. *ACO* works by having agents (*ants*) build solutions step-by-step. These ants are guided by how desirable a path is and the pheromone trails they leave behind [1]. This step-by-step process allows *ACO* to handle strict rules naturally instead of fixing them later. As an ant moves through the network, it keeps track of its load. If a vehicle reaches its maximum capacity, the routing system automatically directs it to return to the depot to finish the route. By following the actual limits of the vehicles and avoiding complex fixes, *ACO* offers a much simpler and more reliable way to solve this optimisation problem.

2.3 Heuristic Implementations

To optimise the routing patterns, the standard *ACO* heuristic was replaced with a custom evaluation function combining **Clarke-Wright distance savings** with an **angular sweep penalty**. Rather than simply prioritising the nearest unvisited node, the algorithm calculates the distance saved by combining two stores into a single route relative to the active warehouse.

$$S_{ij} = d_{w,i} + d_{w,j} - d_{i,j}$$

To prevent the generation of disjointed, *zig-zagging* routes, this savings value is penalised by the absolute angular difference $(\Delta\theta)$ in radians between the two nodes. The final desirability is calculated as:

$$S_{ij}/(1 + \Delta\theta)$$

The addition of 1.0 to the denominator prevents a `DivisionByZero` error if two candidate stores share an identical polar trajectory from the warehouse.

Furthermore, to ensure the generated routing sequences remained geometrically sound, a **2-Opt** local search mechanism was implemented. A completed vehicle route is represented as a one-dimensional array of indices beginning and terminating at the designated warehouse. For example, $[23, 15, 6, 8, 23]$ denotes a vehicle leaving Warehouse 1, visiting three stores, and returning. Due to the probabilistic nature of the ant agents, initial routes may contain intersecting paths. The **Triangle Inequality** implies that intersecting lines on a 2D Euclidean plane will produce a longer total distance than a non-intersecting configuration. The 2-Opt function scans the route array for any two crossing edges where reversing the internal sub-route (e.g., flipping the visitation order from $[15, 6]$ to $[6, 15]$) yields a shorter combined distance. This iterative untangling ensures the *pheromone* matrix is updated with optimal, non-intersecting sub-routes.

2.4 Algorithm Implementation and Results

Constraint Handling:

Ant Colony Optimisation uses a constructive approach, allowing it to manage different vehicle capacities (Vans = 5 stores, Lorries = 18 stores) during route planning, rather than relying on retroactive repair functions. The algorithm keeps track of each route's payload. If a path reaches the maximum capacity of a Lorry, the system forces the route back to the warehouse.

To ensure accuracy in both design and function, the architecture clearly separates the algorithmic agents (*ants*) from the physical fleet (*vehicles*):

- **Ant (The Algorithmic Agent):** An ant does not represent a single vehicle; rather, it acts as a *fleet manager* constructing a complete daily strategy. A single ant will sequentially deploy and route multiple vehicles until all 23 stores are serviced, at which point it evaluates the total combined cost of the entire fleet to inform its *pheromone* deposit.
- **Vehicle (The Capacitated Sub-Route):** The Vans and Lorries represent the physical, constrained resources allocated by the ant. A vehicle's route is simply a single, closed loop originating from a warehouse, visiting a limited subset of stores, and returning, functioning as one isolated component of the ant's overarching master plan.

```
# Constructive constraint enforcement during sub-route generation
if len(route) - 1 == LORRY_CAPACITY:
    valid_nodes = [warehouse] # Forces immediate return to warehouse to
    close the sub-route

```

Listing 3: Agent logic for handling vehicle constraints without the need for retroactive route repair.

Hyperparameter Selection:

The stability of the optimal solution depends on carefully adjusting the ACO hyperparameters. The heuristic importance (β) was set much higher than the pheromone importance (α) to make sure the ants focus on the custom *Clarke-Wright* and *angular sweep* heuristic instead of blind swarm behaviours in the early stages. Additionally, a test showed that increasing the colony to 50 ants led to *Incremental Stagnation*. This happened because too many overlapping, slightly improved pheromone trails confused the swarm. A colony of 40 ants with a 10% evaporation rate worked well for Elitist Updates.

```
num_ants = 40
generations = 100
# Importance of pheromones
alpha = 1.0
# Importance of the heuristic (Set high to respect the Savings/Sweep)
beta = 3.0
evaporation = 0.1  # Pheromone decay rate

```

Listing 4: The optimal hyperparameters to prioritise heuristics and prevent incremental stagnation within the swarm.

Final Results:

The optimisation framework achieved a minimum daily distribution cost of £1118.29. The algorithm capitalised on the efficiency of *Stem Miles* by fully maximising a single Lorry (£3.00/mile) from Warehouse 1 to sweep the bulk of the network, dispatching only a single Van (£2.00/mile) from Warehouse 2 to clear the remaining outlier cluster.

Optimised Configuration (Figure 3):

- **Total Daily Cost:** £1118.29
- **Route 1 (Lorry,** £903.51**):**

$$W1 \to 17 \to 8 \to 15 \to 6 \to 13 \to 18 \to 5 \to 2 \to 4 \to 10 \to 22 \\ \to 9 \to 11 \to 19 \to 12 \to 1 \to 20 \to 14 \to W1$$

- **Route 2 (Van,** £214.78**):** $W2 \to 7 \to 16 \to 3 \to 21 \to 23 \to W2$

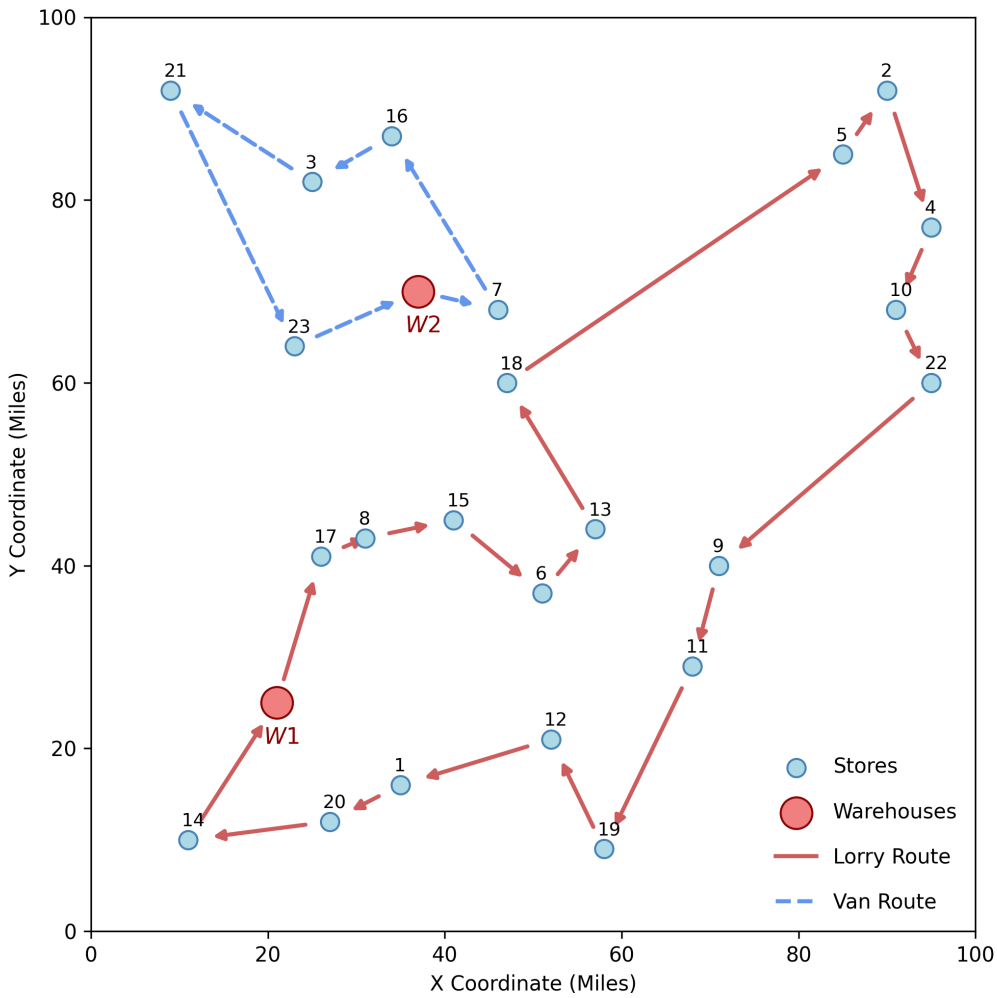


Figure 3: The optimised distribution network. Directional paths denote the 18-store Lorry route (red) from Warehouse 1 and the 5-store Van route (blue) from Warehouse 2.

3 Ariadne Grid Optimization

3.1 Background: The Ariadne System

In classical mythology, the labyrinth of Crete was navigated successfully using *"Ariadne's thread."* This is a lifeline given by someone outside the maze to help a traveller find their way. This idea inspired the Ariadne navigation system [2], which is designed for Unmanned Ground Vehicles (UGVs) operating in **GPS-denied environments**, like in disaster zones or search-and-rescue missions. Since the UGV lacks internal localisation, an aerial drone (UAV) serves as the outside observer and provides a top-down view of the area. A neural network processes the images from the drone, separating the safe paths from obstacles. The system then uses the A^* algorithm to create a safe route to the target, similar to weaving a thread.

While the neural network processes visual information, a key challenge arises when converting the aerial image into a simple grid for the A^* algorithm. To calculate routes in real-time, the environment must be scaled down to a lower-resolution matrix $(R \times R)$. However, choosing the right scale is tricky: a grid that is too **coarse** may block important pathways, while a grid that is too **fine** can cause significant delays in computation.

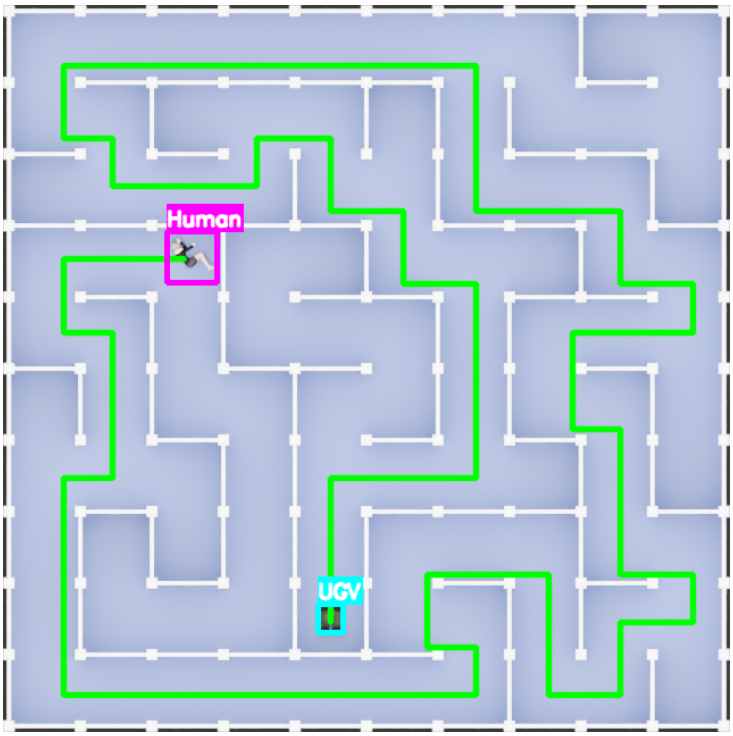


Figure 4: Visual output of the Ariadne system. The aerial image is translated into a matrix, allowing the A^* algorithm to generate an optimal trajectory (green) while avoiding structural boundaries (white).

3.2 Aim: Balancing Latency and Path Feasibility

The primary objective of this optimisation is to determine the **ideal grid resolution**, R, that minimises the total navigation penalty for the Ariadne system. The aerial image is turned into an $R \times R$ matrix, which requires balancing two competing factors:

- **Latency:** the time it takes for the algorithm to compute.
- **Path occlusion rate:** the probability of *spatial aliasing* artificially blocking physically navigable routes.

Note that a **wall-hugging penalty** is intentionally excluded from this objective, as the A^* algorithm functions solely as a global pathfinder; managing physical boundary buffering is assigned to the UGV's local planner and sensory suite.

The following **constraints** ensure the optimization results are practical for edge computing:

- **Integer constraint** $(R \in \mathbb{Z}^+)$**:** Grid resolutions must be whole numbers; fractional cells are invalid.
- **Lower Bound** $(R \geq 10)$**:** The synthetic test environments (mazes) are procedurally generated upon a 10×10 grid. Resolutions below 10 break the layout and block all paths (100% path occlusion rate).
- **Upper Bound** $(R \leq 100)$**:** The theoretical maximum resolution is a $1 : 1$ pixel mapping $(R = 640)$. However, due to the $O(V \log V)$ time complexity of the A^* algorithm, traversing over $400,000$ $(V = 640 \times 640)$ vertices introduces severe latency that violates real-time edge processing constraints.

3.3 Model: The Objective Function

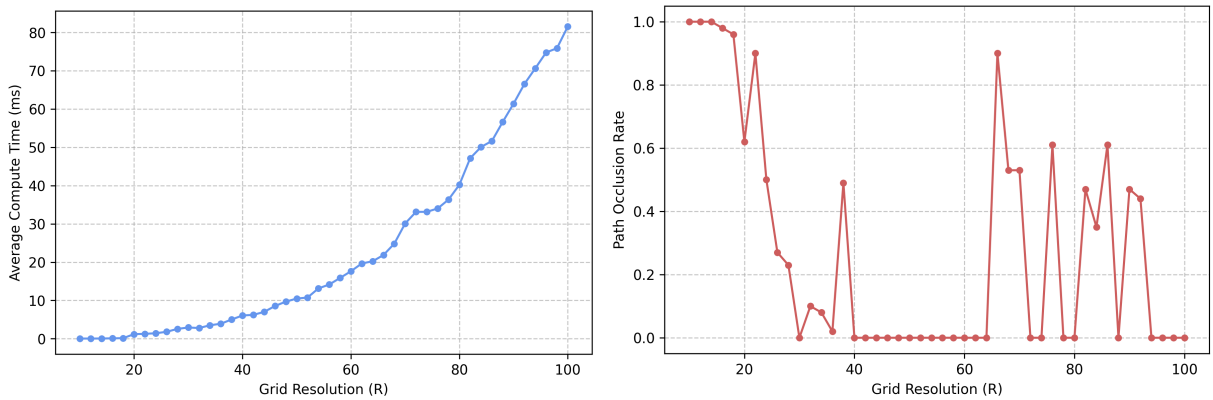
Running the full perception and search systems during optimisation requires too much computing power. To resolve this, the evaluation process was separated from live execution. Instead, a dataset was pre-computed by testing 100 simulated mazes across resolutions from 10 to 100, increasing in steps of 2 $(R \in [10, 100]$ at $\Delta R = 2)$.

The algorithm uses this dataset to balance two competing metrics:

- **Compute Latency** (C_{time})**:** As observed in Figure 5a, latency exhibits predictable, continuous growth tracking the theoretical $O(V \log V)$ time complexity of the A^* algorithm.
- **Path Occlusion** (C_{occ})**:** As shown in Figure 5b, the failure rate is highly unstable. While paths are completely clear at $R = 30$, severe alignment issues emerge at higher resolutions (e.g., a 90% failure spike at $R = 66$). This happens because the grid misaligns with the map structure, causing the calculated path to accidentally clip into obstacles.

Because computation time is measured in milliseconds and occlusion is a probability, combining them directly creates a mathematical imbalance. Without normalisation, the system would prioritise saving fractions of a millisecond over avoiding actual collisions. To fix this, computation time is scaled to a 0.0 to 1.0 range $(\hat{C}_{time})$ to match the occlusion metric (C_{occ}). This ensures the adjustable weights (W) accurately control the trade-off in the final cost function:

$$J(R) = W_{time} \cdot \hat{C}_{time}(R) + W_{occ} \cdot C_{occ}(R)$$



(a) Computational latency cost

(b) Path occlusion rate

Figure 5: Both metrics are stable when the resolution (R) is between 40 and 64. Outside this range, the system fails either because low resolutions block the path or high resolutions cause grid misalignment.

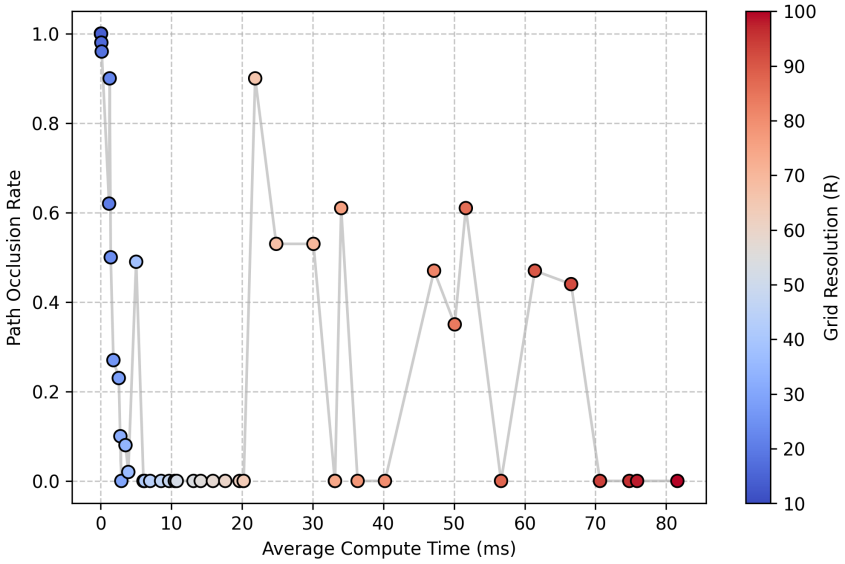


Figure 6: A comparison showing the trade-off between computation time and path occlusion. The colour gradient (ΔR) illustrates the system's unpredictable performance, particularly the tendency for high-resolution setups to cluster into failure states.

3.4 Optimisation Method: Simulated Annealing

To find the optimal balance without getting stuck in failure zones, a **Simulated Annealing** *(SA)* algorithm is used. *SA* introduces controlled randomness, temporarily accepting slower performance to escape unstable areas and find the best overall resolution. The algorithm operates in four main stages:

1. **Neighbourhood Search:** The system searches within a set resolution range $(R \in [10, 100])$, moving in small steps $(\Delta R = \pm 2)$. Larger steps are avoided so the process does not become completely random.

2. **Weighted Evaluation:** Finding an open path is prioritised over minor speed improvements. Avoiding blocked paths gets a 0.6 weight (W_{occ}), while calculation time gets a 0.4 weight (W_{time}).

3. **Probability Acceptance:** If a new step results in a worse score $(\Delta E > 0)$, it might still be accepted based on a probability formula $(P = e^{-\Delta E/T})$. A cooling factor of 0.95 gradually lowers this chance, forcing the system to eventually lock in on the best solution.

4. **Multi-Start Verification:** To prevent random variations from stopping the search too early, the entire loop runs five times to guarantee a reliable result.

3.5 Results

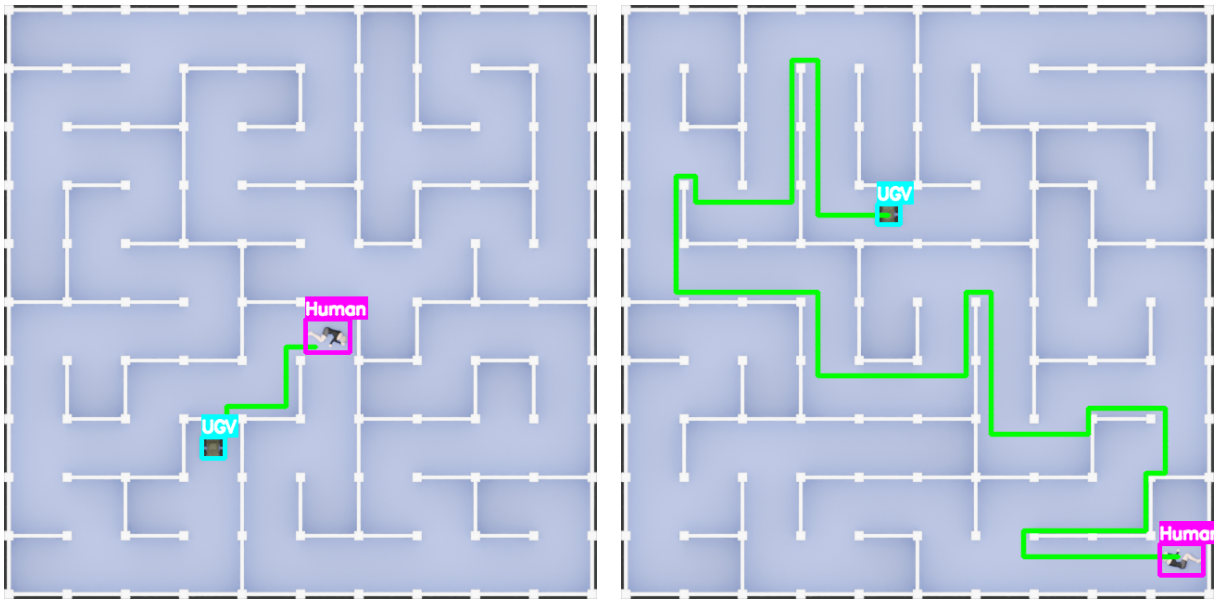
- **The Mathematical Best:** $R = 30$ is the fastest successful resolution (2.92 ms, 0% failures), but it is too close to a failure zone. Dropping to $R = 28$ results in a 23% failure rate.

- **The Safe Zone:** The data shows a stable plateau of zero failures stretching from $R = 40$ to $R = 64$.

- **The Practical Choice:** For a real-world UGV, reliability outweighs extreme speed. Setting the resolution to the centre of the stable zone $R = 52$ (10.76 ms, 0% failures) protects the system from both low-resolution blocked paths and high-resolution clipping errors.

3.6 Conclusion

This project showed that both high and low grid resolutions cause distinct failures (Figure 7). These failures prove that a global pathfinder (UAV) cannot operate alone, as it only understands the grid, not the physical world. Therefore, the UGV must use real-time sensors (like LiDAR or cameras) for immediate obstacle avoidance. The calculated path serves only as a general guide, while the active sensors ensure physical safety.



(a) Missing Geometry (Low Resolution) (b) Path Clipping (High Resolution)

Figure 7: At **low resolutions** thin walls disappear from the grid, causing the system to treat solid obstacles as open space. At **high resolutions** the calculated route hugs corners too tightly, causing the path to clip into walls.

References

[1] S. Luke, *Essentials of Metaheuristics*. Lulu, second ed., 2013. Available for free at https://people.cs.gmu.edu/∼sean/book/metaheuristics/.

[2] O. Oyedeji, "Ariadne: A synthetic proof of concept for air-to-ground optical navigation." GitHub repository, April 2026. Unpublished coursework, ITNPAI2: AI for Computer Vision. [Online]. Available: `https://github.com/Daniel-Ope06/ariadne-poc`.